

ProHactive

Comprehensive Product Requirements Document

ProHactive Product Requirements Document (PRD)

Comprehensive project brief intended for developers, agencies, technical cofounders, and AI coding agents.

1. Product Vision

ProHactive is a proactive productivity platform built around time-blocking rather than traditional task lists. The goal is to help users transform intentions into executable daily schedules.

2. Problem Statement

Many productivity tools collect tasks but do not help users realistically allocate time. Users end up with long task lists, poor prioritization, and frequent planning failures.

3. Target Users

Primary users are students, solo professionals, freelancers, developers, and knowledge workers who want a structured daily planning system.

4. Core Principles

Time blocks are the primary object. Tasks should be scheduled. Planning should be simple. Rescheduling should be frictionless. The system should encourage execution over task accumulation.

5. MVP Features

• Task creation and editing • Priority levels • Daily planner • Drag-and-drop scheduling • Schedule review • Task completion tracking • Responsive PWA • Local and backend persistence

6. Screen Specifications

Dashboard: overview of day. Task Manager: create, edit, prioritize tasks. Daily Planner: timeline and time blocks. Review Screen: completed vs incomplete tasks. Settings: preferences and account settings.

7. Frontend Architecture

React + TypeScript + Vite. Feature-based architecture. Zustand state management. Reusable UI components. PWA support. Responsive-first design.

8. Suggested Folder Structure

src/ app/ features/ tasks/ planner/ schedule/ analytics/ components/ hooks/ services/ store/ utils/ types/

9. Scheduling Engine

Separate from UI. Input: tasks, priorities, durations, constraints. Output: recommended daily schedule. Must be independently testable and replaceable.

10. State Management

Global Zustand store. Task store. Planner store. Settings store. Persistence middleware. Selectors for performance.

11. Backend Evolution

MVP may start frontend-first. Future stack: Node.js Express/NestJS PostgreSQL Authentication Cloud deployment.

12. Database Model

Users Tasks Schedules TimeBlocks Settings AnalyticsEvents Relationships: User -> Tasks User -> Schedules Schedule -> TimeBlocks

13. API Design

POST /tasks GET /tasks PUT /tasks/:id DELETE /tasks/:id POST /schedule/generate GET /schedule/:date GET /analytics

14. PWA Requirements

Installable application. Offline support. Local caching. Fast loading. Mobile-first experience.

15. AI Roadmap

Phase 1: Manual scheduling. Phase 2: Schedule recommendations. Phase 3: AI prioritization. Phase 4: Adaptive planning engine. Phase 5: Personalized productivity coach.

16. Post-MVP Features

Calendar integrations. Habit tracking. Focus sessions. Analytics dashboard. Notifications. Cross-device sync. Native mobile apps.

17. Build Order

1. Design system 2. Task management 3. Zustand store 4. Daily planner 5. Drag-and-drop scheduling 6. Persistence 7. Review screen 8. PWA support 9. Backend integration 10. AI layer preparation

18. Success Metrics

Daily active usage. Tasks scheduled. Tasks completed. Schedule adherence. User retention. Planning accuracy.

19. Non-Functional Requirements

Maintainable code. Type-safe architecture. Scalable design. Performance optimized. Accessible UI. Testable business logic.

20. Final Objective

Build a premium productivity platform whose core differentiator is proactive schedule construction, intelligent planning, and eventual AI-assisted execution management.